

FAISS 三种向量检索方式学习

FAISS (Facebook AI Similarity Search) 是一个高效的向量检索库，特别适用于大规模高维数据的相似度检索。它的核心原理是通过不同类型的索引结构来加速相似度的搜索过程。三个基础索引内容是：

1. IndexFlatL2
2. IndexIVFFlat
3. IndexIVFPQ

1. IndexFlatL2

原理：

最简单的索引类型，可以被看作是一个“暴力搜索”方法，它直接对所有的向量进行精确搜索，计算每个查询向量与数据集中所有向量的 L2 距离 (欧几里得距离)。

优点：

- **精确**：返回的结果是精确的，即在给定的查询向量中，最接近的向量会被精确地返回。
- **简单实现**：由于没有任何近似方法，实现简单，计算和存储开销低。

缺点：

- **计算开销高**：对于大型数据集，计算每个查询与所有数据的 L2 距离非常慢。因为每次查询都需要扫描所有向量。

 适用场景：

数据集较小，对性能要求低，可以接受线性扫描的计算开销。

2. IndexIVFFlat

原理：

基于倒排文件 (Inverted File) 的近似算法。在这个索引中，首先将所有的向量分成多个簇，每个簇代表了一组相似的向量。簇的划分是基于 K-means 聚类 或类似的聚类算法来实现的。每个簇会有一个“簇中心”，然后将这些簇的中心作为倒排索引的条目。

在查询时，IndexIVFFlat 会首先通过查询向量与各个簇的中心进行匹配，找到最近的簇，然后在这个簇中执行精确的 L2 搜索，而不需要在所有向量中进行全局搜索。

工作流程：

1. **训练阶段**：对所有向量进行 K-means 聚类，确定每个簇的中心。
2. **索引阶段**：对于每个向量，记录它属于哪个簇。
3. **查询阶段**：
 - 查询向量首先计算与各个簇中心的距离，选择距离最近的簇。
 - 在选定的簇中，执行精确的 L2 搜索，返回最接近的向量。

优点：

- **近似搜索**：由于簇划分的关系，返回的结果是近似的，不保证全局最近邻向量。
- **需要训练**：IndexIVFFlat 需要先进行 K-means 聚类 训练，这可能增加时间成本。

缺点：

- **效率提升**：通过划分簇，减少了需要计算的距离数量，使得查询速度大大提升。

- **可扩展性**：在大规模数据集上的性能更好，尤其是在搜索时只访问部分簇而非整个数据集。

🍰 适用场景：

中等规模的数据集，能够接受一定的近似误差，但要求较高的查询速度。

3. IndexIVFPQ

原理：

基于 **倒排文件 (IVF)** 和 **量化 (Product Quantization, PQ)** 技术的组合索引。它在 **IndexIVFFlat** 的基础上引入了量化 (PQ)，进一步降低了存储需求并加速了查询。

- **倒排文件**: 与 IndexIVFFlat 一样，首先将数据集划分为多个簇，每个簇有一个中心。
- **量化 (Product Quantization, PQ)**：每个簇内部的向量进一步被量化。具体来说，**PQ** 将向量分为多个子向量，并对每个子向量进行量化。**每个子向量使用有限的码字表进行近似，从而减少存储需求。**

🌟 通过使用量化技术，**IndexIVFPQ** 大幅减少了存储空间和内存占用，同时在查询时通过量化码字的查找可以更高效地进行相似度计算。

工作流程：

1. 训练阶段：

- 执行 **K-means 聚类**，划分簇。
- 对每个簇内部的向量进行 **Product Quantization (PQ)**，即将每个向量拆分为多个子向量，并对每个子向量进行量化。

2. 索引阶段：

- 为每个向量生成一个 **量化的编码**，存储在倒排索引中。

2. 查询阶段：

- 查询向量首先计算与簇中心的距离，找到最近的簇。
- 在选定的簇中，使用量化编码进行快速近似查找。

优点：

- **更低的存储开销**：通过量化，显著降低了内存占用和存储需求，适合大规模数据集。
- **查询效率高**：相比于 **IndexIVFFlat**, **IndexIVFPQ** 更节省存储和内存，因此在大规模向量搜索时效率更高。

缺点：

- **近似误差**：由于量化引入了误差，返回的结果是近似的，精度可能低于精确搜索。
- **训练复杂度**：量化技术和倒排索引的结合可能使得训练过程比 **IndexIVFFlat** 更为复杂。

🍰 适用场景：

大规模数据集，要求存储和内存占用低，同时能接受一定的近似误差和训练成本。

索引类型	精度	查询速度	存储需求	训练成本	适用场景
<code>IndexFlatL2</code>	精确	慢（暴力搜索）	高（无压缩）	低	小规模数据集，精确查询
<code>IndexIVFFlat</code>	近似	较快（簇划分加速）	中等（簇中心存储）	中等	中规模数据集，要求较快查询
<code>IndexIVFPQ</code>	近似	非常快（量化加速）	低（量化存储）	高	大规模数据集，存储和内存有限

4. `IndexIVFFlat` 向量检索原理与检索过程

4.1 基本理解

`IndexIVFFlat` 是 FAISS 中常用的一种向量索引方式。

可以理解为：

先用 K-means 把向量分组，再只在相关分组中进行精确搜索。

它由两部分组成：

组成	作用
IVF	倒排文件，用于将向量按簇分组，减少搜索范围
Flat	在候选簇内进行原始向量的精确距离计算

`IndexIVFFlat` = K-means 聚类分桶 + 候选簇内暴力搜索

4.2 示例场景

假设有 10 个二维向量，每个向量表示一个点：

```
(1.0, 2.0)
(2.0, 3.0)
(3.0, 3.5)
(8.0, 8.0)
(9.0, 8.5)
(10.0, 9.0)
(5.0, 6.0)
(6.0, 5.5)
(7.0, 7.0)
(6.5, 6.5)
```

现在希望通过 `IndexIVFFlat` 加速相似度查询。

4.3 K-means 聚类

首先使用 `K-means` 将这些向量划分为 2 个簇。

簇	包含的向量	簇中心
簇 1	(1.0, 2.0)、(2.0, 3.0)、(3.0, 3.5)、(5.0, 6.0)、(6.0, 5.5)、(6.5, 6.5)	(4.5, 5.0)
簇 2	(8.0, 8.0)、(9.0, 8.5)、(10.0, 9.0)、(7.0, 7.0)	(8.5, 8.0)

这一步的作用是：

把相似的向量放到同一个簇中，后续查询时只搜索相关簇。

4.4 倒排索引存储

聚类完成后，FAISS 会为每个簇建立一个倒排列表。

倒排列表中保存属于该簇的向量 ID。

簇	簇中心	倒排索引	对应向量
簇 1	(4.5, 5.0)	[0, 1, 2, 6, 7, 9]	属于簇 1 的向量
簇 2	(8.5, 8.0)	[3, 4, 5, 8]	属于簇 2 的向量

可以理解为：

倒排索引记录的是：
每个簇里面有哪些向量。

4.5 查询检索

假设查询向量为：

`query = (7.5, 7.5)`

目标是找到与它最相似的向量。

4.5.1 先找最近的簇中心

先计算查询向量与每个簇中心的距离：

簇中心	距离
簇 1 中心 (4.5, 5.0)	约 3.91
簇 2 中心 (8.5, 8.0)	约 1.12

因为查询点距离 **簇 2** 更近，所以优先搜索簇 2。

4.5.2 访问簇 2 的倒排列表

簇 2 的倒排索引是：

`[3, 4, 5, 8]`

对应的候选向量是：

```
(8.0, 8.0)
(9.0, 8.5)
(10.0, 9.0)
(7.0, 7.0)
```

这一步避免了遍历全部 10 个向量，只需要检查簇 2 中的候选向量。

4.5.3 在候选簇内计算距离

计算查询向量 (7.5, 7.5) 与簇 2 中每个向量的距离：

候选向量	距离
(8.0, 8.0)	约 0.707
(9.0, 8.5)	约 1.803
(10.0, 9.0)	约 2.915
(7.0, 7.0)	约 0.707

因此，最接近查询向量的是：

```
(8.0, 8.0)
(7.0, 7.0)
```

4.6 检索流程总结

IndexIVFFlat 的检索过程可以总结为：

- 1. 使用 K-means 将所有向量划分为多个簇
- 2. 为每个簇建立倒排列表，记录该簇中的向量 ID
- 3. 查询时，先找到距离 query 最近的簇中心
- 4. 只在相关簇中搜索候选向量
- 5. 在候选向量中计算真实距离
- 6. 返回距离最近的 top_k 个结果

4.7 一句话理解

IndexIVFFlat 不是直接搜索全部向量，而是先通过聚类缩小搜索范围，再在候选簇中进行精确距离计算。

4.8 优缺点

维度	说明
优点	比暴力搜索更快，适合中大规模数据
优点	簇内仍使用原始向量计算，精度较高
缺点	需要提前训练 K-means 聚类
缺点	只搜索部分簇，可能牺牲少量召回率
适用场景	数据规模较大，同时希望在速度和精度之间取得平衡